

Attività svolta: Programmazione PLC con operazioni logiche e aritmetiche

Programmazione:

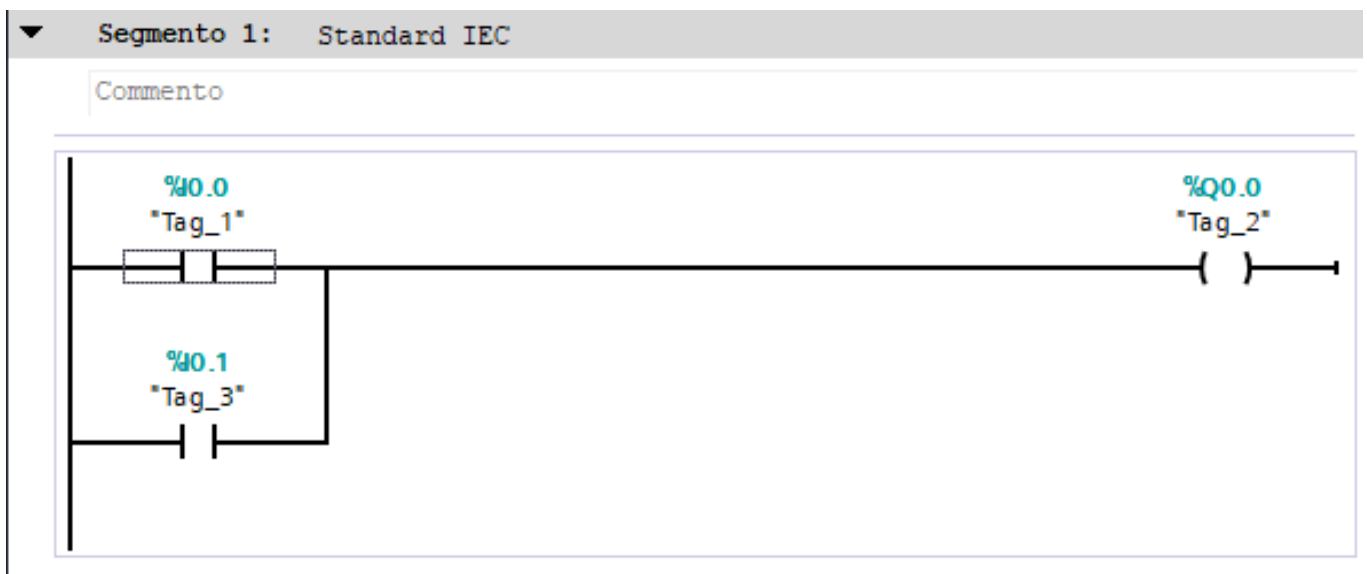
Il blocco “Main [OB1]” è un Organization Block ovvero che richiama le funzioni e il blocco “Blocco_1 [FC1]” è un Functional Block ovvero che dichiara funzioni senza struttura dati. La programmazione può avvenire in 3 modalità:

- A blocchi (Standard IEC)
- In linguaggio SCL
- In linguaggio AWL (Legacy Siemens)

Operazioni logiche booleane:

Bisogna sapere che non può esistere un uscita senza almeno un ingresso...

Programmazione a BLOCCHI:



“Tag_1” e “Tag_3” sono ingressi e “Tag_2” è un uscita. C’è una porta logica tra il Tag_1 e il Tag_2 ovvero l’OR

Programmazione in AWL:

Segmento 2: AWL (Legacy Siemens)			
Commento			
1			
2	A(		
3	O	"Tag_1"	%I0.0
4	O	"Tag_3"	%I0.1
5	)		
6	=	"Tag_2"	%Q0.0
7			

A() è AND e in questo caso è come un IF - **O** è OR - **=** assegna il valore della condizione alla variabile

Programmazione in SCL:

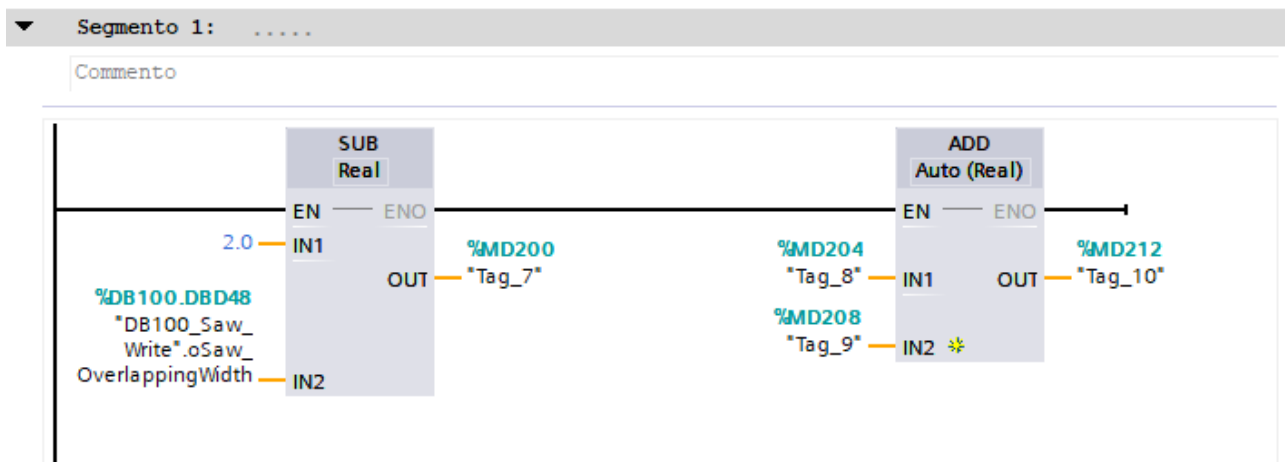
Segmento 3: SCL	
Commento	
1	
2	IF ("Tag_1" OR "Tag_3") THEN
3	"Tag_2" := TRUE;
4	ELSE
5	"Tag_2" := FALSE;
6	END_IF;
7	
8	// oppure
9	
10	"Tag_2" := ("Tag_1" OR "Tag_3");

Simile a C++ ma non si usano le parentesi graffe, si usa **THEN** per aprire e **END_IF** per chiudere e per l'assegnamento non si usa "=" ma ":=". Inoltre si può direttamente assegnare alla variabile (**booleana**) la condizione.

Operazioni aritmetiche:

Si possono effettuare delle operazioni aritmetiche come la somma, radice, media, valore max, etc.. con delle variabili o con delle costanti/valori che inseriamo noi.

Programmazione a BLOCCHI:



Entrambe le funzioni richiedono almeno 2 ingressi e un risultato in uscita:
nel caso della sottrazione abbiamo una costante (2.0) e una variabile (%DB 100...) in ingresso e una variabile (%MD200) in uscita (risultato), mentre nel caso della somma abbiamo solo variabili. La funzione riconosce in automatico il tipo delle variabili e se le variabili sono di tipo diverso tra di loro, la funzione le converte (disegna un blocchetto)

Programmazione in AWL:

▼ Segmento 2:

Commento

1				
2	L	2.0	// Ingresso 1	2.0
3	L	"DB100_Saw_Write".oSaw_OverlappingWidth	// Ingresso 2	%DB100.DB...
4	-R		// Effettuo sottrazione	
5	T	"Tag_7"	// Risultato in uscita	%MD200
6				
7				
8	L	"Tag_8"	// Ingresso 1	%MD204
9	L	"Tag_9"	// Ingresso 2	%MD208
10	+R		//Effettuo somma	
11	T	"Tag_10"	// Risultato in uscita	%MD212
12				
13				

L dichiara l'ingresso - **R** dichiara l'operazione (- **SUB** e + **ADD**) - **T** dichiara dove finisce il risultato in uscita

Programmazione in SCL:

▼ Segmento 3:

Commento

```
1
2  "Tag_7" := 2.0 - "DB100_Saw_Write".oSaw_OverlappingWidth;
3  "Tag_10" := "Tag_8" + "Tag_9";
```

Uguale a C++...